“시간이 아깝지 않은”

SK네트웍스 Family AI캠프 교재8

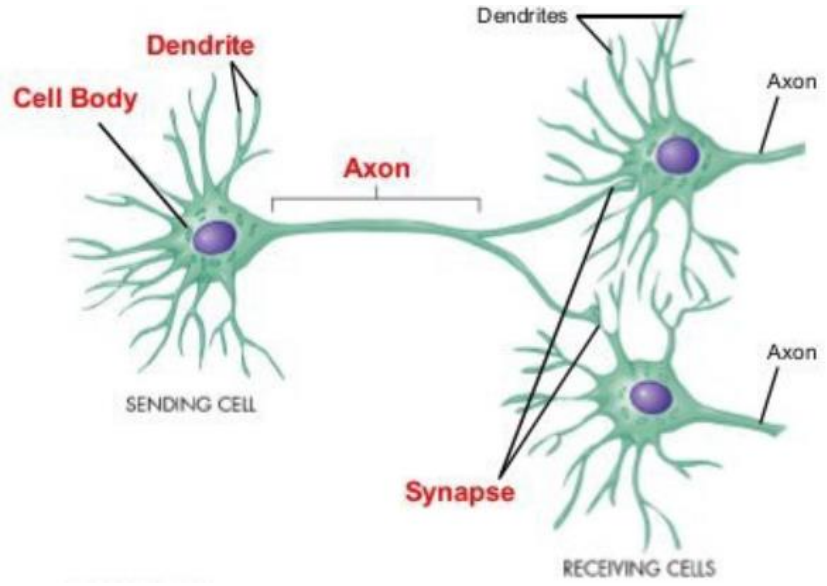
강사 이영배

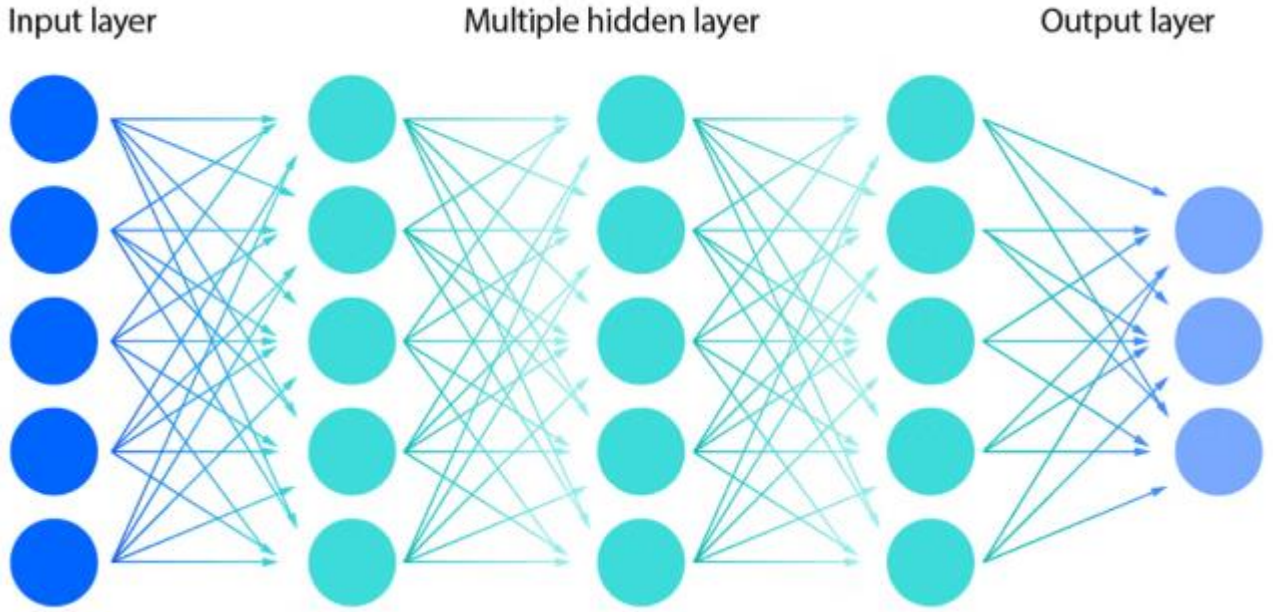
19. 딥러닝(Deep Learning)

딥러닝 개요

- 딥러닝(Deep Learning)의 정의

  : 인간의 뇌 신경망 구조를 모방한 인공신경망(Artificial Neural Network)을 기반으로,
  대량의 데이터에서 스스로 특징을 학습하는 머신러닝의 한 분야

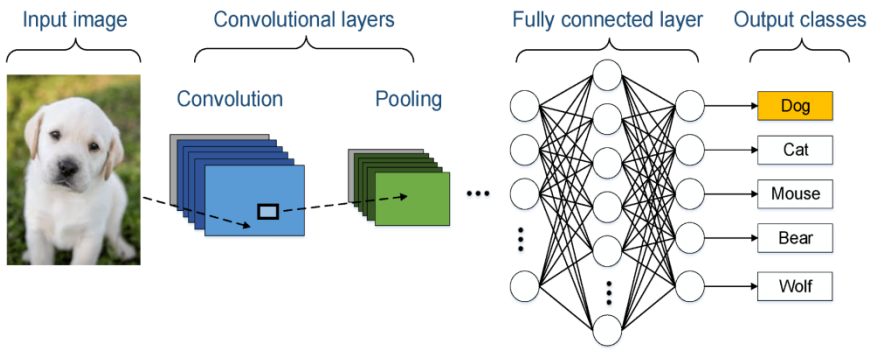




딥러닝 개요 – 머신러닝 vs 딥러닝 비교



sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	label
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
7.0	3.2	4.7	1.4	versicolor
6.4	3.2	4.5	1.5	versicolor
6.9	3.1	4.9	1.5	versicolor
6.3	3.3	6.0	2.5	virginica
5.8	2.7	5.1	1.9	virginica
7.1	3.0	5.9	2.1	virginica



딥러닝 개요 – 머신러닝 vs 딥러닝 비교 정리

비교 항목	머신러닝 (Machine Learning)	딥러닝 (Deep Learning)
특징 추출	사람이 직접 정의 (Feature Engineering)	모델이 데이터로부터 스스로 학습
데이터 의존도	적은 양의 데이터로도 유의미한 모델 성능 확보 가능	데이터가 많을수록 성능이 극대화됨
학습 시간	수 초 ~ 수 시간 내 학습 완료 (빠름)	수 일 ~ 수 주일 이상 소요 (무거움)
연산 자원	일반적인 CPU 기반 연산 가능	GPU / TPU 등의 병렬 연산 장치 필수
대표 알고리즘	결정 트리, 로지스틱 회귀 등	CNN, RNN, LSTM, Transformer, BERT 등

딥러닝 핵심 기술 — 주요 알고리즘

- CNN (Convolutional Neural Network / 합성곱 신경망)
  1) 이미지의 공간적 구조(Spatial Structure)를 유지하며 유의미한 특징을 추출하는 필터 기반 알고리즘
  2) 용도 : 컴퓨터 비전(CV) 분야의 핵심 (이미지 분류, 객체 탐지, 세그멘테이션 등)

- RNN / LSTM (Recurrent Neural Network / 순환 신경망)
  1) 데이터의 순서와 시간적 흐름을 기억하기 위해 은닉층의 출력을 다시 입력으로 순환시키는 구조
  2) 용도 : 시계열 데이터 예측, 초기 자연어 처리(NLP) (영화 리뷰 감성 분석 등)

- Transformer / BERT (트랜스포머 기반 모델)
  1) 순환 신경망의 한계를 극복하고, 문장 내 단어 간의 관계를 동시에 파악하는 Self-Attention 메커니즘 기반
  2) 용도 : 최신 거대 언어 모델(LLM), 고도화된 비정형 텍스트 분석 및 문장 유사도 측정

딥러닝 핵심 기술 — 오픈 프레임워크 비교

- TensorFlow & Keras (구글 주도)
  : 직관적인 코드 구조를 제공하여 입문 및 프로토타이핑에 매우 유리

- PyTorch (메타 주도)
  : 글로벌 표준

퍼셉트론(perceptron)

- 의미

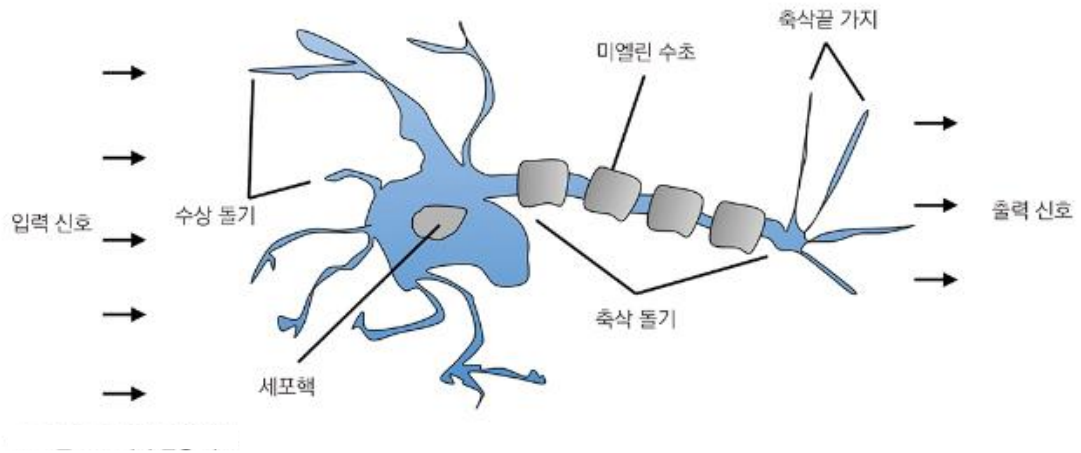
1) perceive → perception

2) 신경세포 → neuron

3) perception + neuron → perceptron(인공 뉴런, 논리 회로)

4) perceptron(인공 뉴런) 정의 : 다수의 입력 신호를 받아서 하나의 신호를 출력하는 인공 뉴런

5) 가중치(weight) : 뉴런의 출력 신호를 낼 지 말지를 결정하기 위해 입력 신호에 곱하는 계수 → 신경세포체에서 합쳐진 신호가 특정 임계 값을 넘을 지 말지를 결정

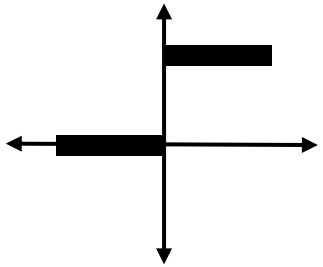


$$y = w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_nx_n + b$$

퍼셉트론(perceptron)의 수학적 정의

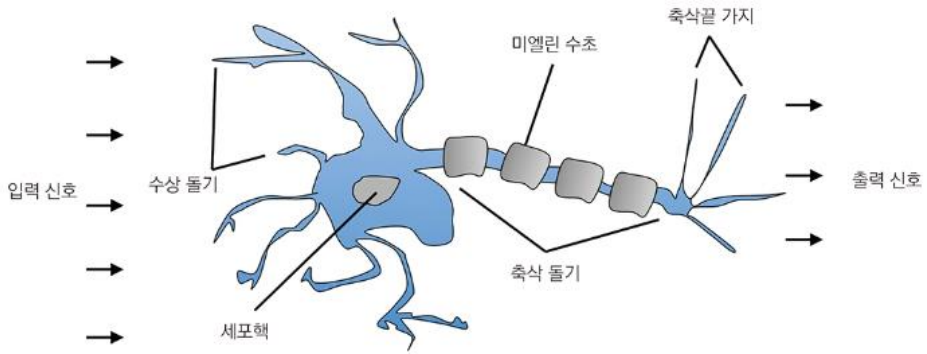
- 다수의 입력 신호(x1, x2)를 받아 하나의 신호(y)를 출력하며, 기존 수식은 임계값(theta)을 기준으로 작동했으나, 현대 연산의 표준에 맞춰 편향(b)을 도입한 수식으로 정의한다

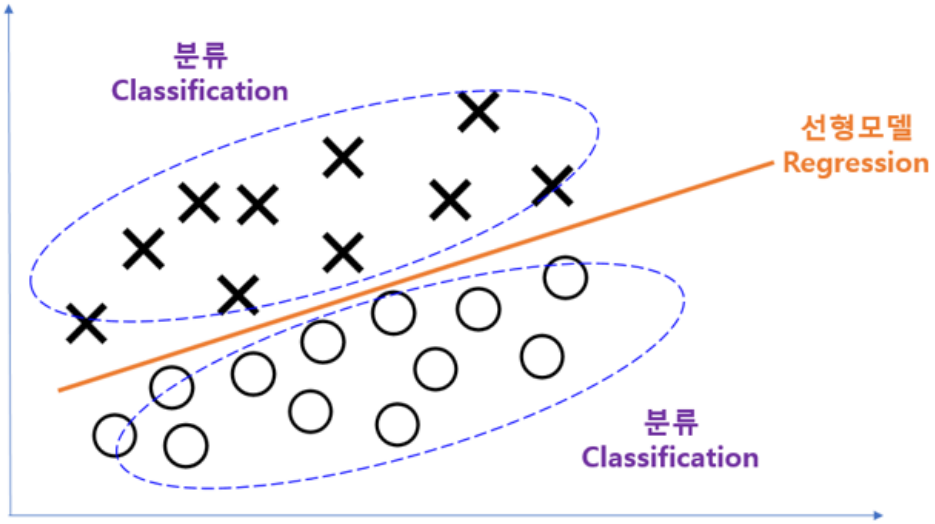
$$z = \begin{cases} 0 & (w_1x_1 + w_2x_2 < \theta) \\ 1 & (w_1x_1 + w_2x_2 >= \theta) \end{cases} \Rightarrow z = \begin{cases} 0 & (-\theta + w_1x_1 + w_2x_2 < 0) \\ 1 & (-\theta + w_1x_1 + w_2x_2 >= 0) \end{cases} \Rightarrow z = \begin{cases} 0 & (b + w_1x_1 + w_2x_2 < 0) \\ 1 & (b + w_1x_1 + w_2x_2 >= 0) \end{cases}$$



퍼셉트론(perceptron)의 수학적 정의

- 퍼셉트론은 결국 0이냐 1이냐를 맞추는 이진 분류기이며, 이진 분류의 '결정 경계(Decision Boundary)'로 작동하는 선형 분류 모델이라 할 수 있다.





입력 신호의 총합

$$z = w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_nx_n + b$$

퍼셉트론(perceptron)의 의의와 필요성

- 딥러닝의 구조적 모태 (Structural Foundation)

1) 빌딩 블록(Building Block)의 역할: 벽돌 하나하나가 모여 거대한 빌딩을 이루듯, 퍼셉트론 수십억 개가 유기적으로 연결된 것이 현대의 심층신경망(Deep Neural Network)이다.

2) 거대 아키텍처 이해의 출발점: 인공 뉴런(Perceptron) 하나가 작동하는 방식을 모르면, 이것이 수천만 개 쌓인 CNN이나 Transformer(ChatGPT의 기반 기술)의 연산 과정을 이해할 수 없다.

논리 게이트(Logic Gate)

- 개념

  1) 논리 게이트는 컴퓨터의 두뇌(CPU)를 구성하는 가장 작은 단위의 '스위치'

  2) 입력된 데이터들이 특정 '조건(규칙)'을 만족할 때만 신호를 통과시키거나 차단하기 때문에 '논리(Logic)' 게이트라고 부른다.

- '게이트(Gate, 문)'라고 표현하는 이유

  : 흐르는 물을 막거나 통과시키는 '수문(Water Gate)'처럼, 전기 신호를 흐르게 할지(1, True) 혹은 멈추게 할지(0, False) 결정하는 문 역할을 하기 때문이다.

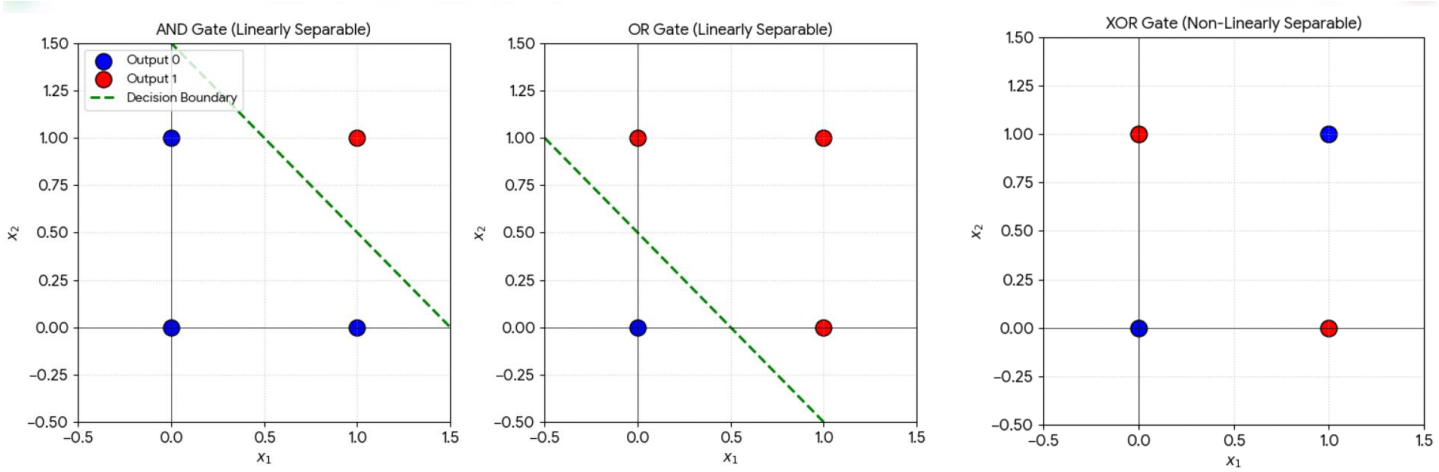
논리 게이트(Logic Gate)

- 논리 게이트 종류:
  1) AND (논리곱): 모든 입력이 1일 때만 1을 출력
  2) OR (논리합): 입력 중 하나라도 1이면 1을 출력
  3) NAND (부정논리곱): AND 연산 결과를 뒤집은 것. 모든 입력이 1일 때만 0을 출력
  4) XOR (배타적논리합): (0, 1) 또는 (1, 0)과 같이 입력값이 서로 다를 때만 1을 출력

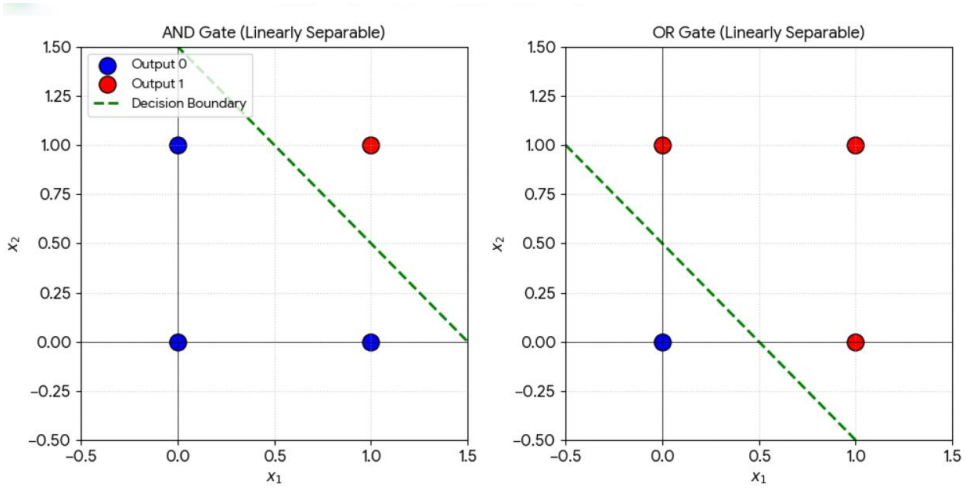
논리 게이트(Logic Gate)

- 논리 게이트 진리표: 논리 게이트에 들어올 수 있는 모든 입력 신호의 조합과 그에 따라 결정되는 출력 신호의 결과를 빠짐없이 정리한 표

입력 x1	입력 x2	AND 출력	NAND 출력	OR 출력	XOR 출력
0	0	0	1	0	0
1	0	0	1	1	1
0	1	0	1	1	1
1	1	1	0	1	0



퍼셉트론을 이용한 AND / OR 게이트 구현



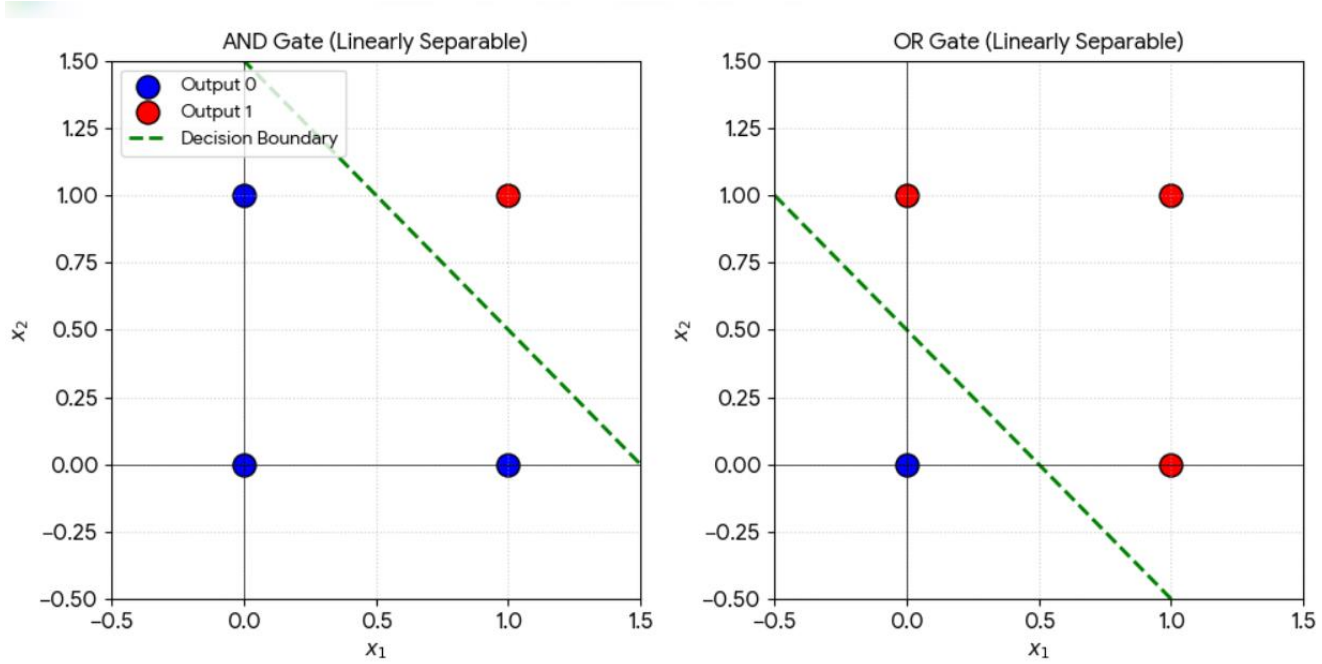
0.5x1+0.5x2-0.7=0

0.5x1+0.5x2-0.2=0

단층 퍼셉트론의 한계와 다층 퍼셉트론(MLP)

- 선형 분리 가능과 불가능

1) 선형 분리 가능: AND, NAND, OR 게이트는 2차원 좌표평면 상에 네 가지 입력 조합을 점으로 찍었을 때, 하나의 직선(결정 경계)을 긋는 것만으로도 출력이 0인 영역과 1인 영역을 완벽하게 나눌 수 있으며, 이를 '선형 분리'가 가능하다고 한다.



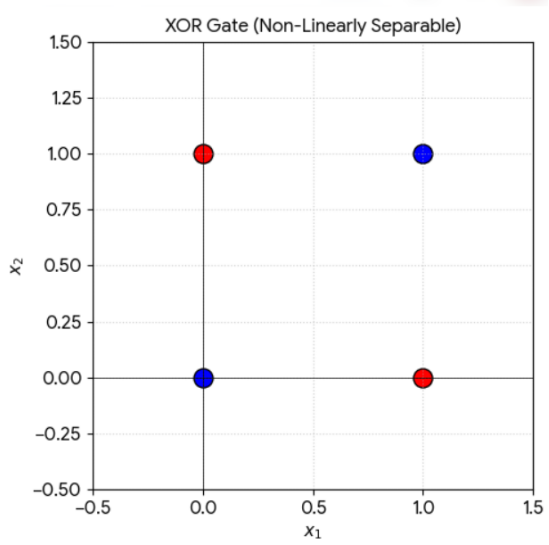
단층 퍼셉트론의 한계와 다층 퍼셉트론(MLP)

- 선형 분리 가능과 불가능

2) XOR 게이트의 난제 (선형 분리 불가능)

① XOR 게이트의 데이터 분포를 평면에 시각화해 보면 (0,0)과 (1,1) 위치에는 0이 찍히고, (1,0)과 (0,1) 위치에는 1이 찍혀 대각선 방향으로 서로 엇갈려 있다. 이 구조에서는 단 하나의 '직선'으로는 0과 1을 절대로 분리할 수 없다.

② 퍼셉트론 식(w1x1 + w2x2 + b = 0)이 수학적으로 '직선의 방정식'을 의미하기 때문에, 퍼셉트론 하나만으로는 XOR 문제를 구조적으로 절대 해결할 수 없다는 한계에 직면하게 된다



단층 퍼셉트론의 한계와 다층 퍼셉트론(MLP)

- 왜 XOR 문제가 인공지능 역사에서 가장 중요한가? (첫 번째 AI 겨울)
  1) 인공지능을 매장했던 수학적 증명: 1969년, 당시 컴퓨터 과학의 거두였던 마빈 민스키 교수는 저서 《퍼셉트론(Perceptrons)》을 통해 "단층 퍼셉트론은 이 간단한 XOR 문제조차 풀 수 없으며, 레이어를 쌓아 다층으로 만든다 해도 이를 학습시킬 방법은 존재하지 않는다"고 수학적으로 비판했음
  2) 제1차 인공지능의 겨울 (AI Winter): 이 강력한 학술적 사망 선고로 인해, 전 세계 정부와 기업의 인공지능 연구 지원이 일제히 끊겼고, 수많은 연구자가 학계를 떠나며 인공지능 연구는 약 20년간 암흑기에 빠지게 되었음
  3) 'Deep'의 필연적 탄생: 그러나 훗날 연구자들은 민스키 교수의 한계를 비웃듯 층을 여러 개 쌓는 '다층 퍼셉트론(MLP)'과 오차를 뒤로 보내며 스스로 매개변수를 고치는 '오차역전파' 알고리즘을 세상에 내놓으며 이 난제를 극복했음
  4) 즉, XOR 문제는 단순한 수학 퀴즈가 아니라, 인공지능을 한 번 죽음에 이르게 했다가 현대의 강력한 '심층(Deep)' 신경망으로 부활시킨 딥러닝 역사상 가장 위대한 터닝 포인트이라고 할 수 있음

단층 퍼셉트론의 한계와 다층 퍼셉트론(MLP)

- 다층 퍼셉트론 (Multi-Layer Perceptron)을 통한 한계 극복

1) '층'을 쌓아 문제 해결

① 출력층 앞에 은닉층(Hidden Layer)을 도입

② 구조: 입력층 → 은닉층 →출력층의 다층 구조

2) 게이트의 유기적 조합

$$\text{Input } (x_1, x_2) \longrightarrow \begin{cases} \text{NAND} \rightarrow s_1 \\ \text{OR} \rightarrow s_2 \end{cases} \longrightarrow \text{AND}(s_1, s_2) \longrightarrow \text{Output } (y)$$

① 은닉층에 NAND와 OR 게이트를 병렬로 배치: 입력층의 데이터 (x1, x2)가 두 게이트에 동시에 입력됨

② 은닉층 노드 1(h1): 두 입력에 대해 NAND 연산을 수행 (값이 같으면 0, 다르면 1)

③ 은닉층 노드 2(h2): 두 입력에 대해 OR 연산을 수행 (하나라도 1이면 1)

④ 출력층 노드 (y): 은닉층의 결과물 (h_1, h_2)를 입력 받아 AND 연산 수행 → 문제 해결

단층 퍼셉트론의 한계와 다층 퍼셉트론(MLP)

- 다층 퍼셉트론 (Multi-Layer Perceptron)을 통한 한계 극복

입력 X1	입력 X2	은닉층 H1 (NAND)	은닉층 H2 (OR)	출력층 Y (AND)	최종 결과 (XOR)
0	0	1	0	0	**0**
0	1	1	1	1	**1**
1	0	1	1	1	**1**
1	1	0	1	0	**0**

퍼셉트론 실습

활성화 함수의 개념과 필요성

- 활성화 함수(Activation Function)의 정의

1) 입력 신호의 총합(w1x1 + w2x2 + b)을 받아, 다음 뉴런으로 보낼 출력 신호의 형태로 변환하고 결정하는 함수이다.

2) 앞선 단원에서 퍼셉트론 수식의 계산 결과를 0 또는 1로 최종 변환해주던 연산 단계(조건문)가 바로 활성화 함수의 한 종류이다(계단 함수).

```
def AND(x1, x2):
    x = np.array([x1, x2])
    w = np.array([0.5, 0.5])
    b = -0.7
    tmp = np.sum(x * w) + b
    return 1 if tmp > 0 else 0
```

활성화 함수의 개념과 필요성

- 딥러닝에서 활성화 함수가 반드시 필요한 이유

1) '망'의 깊이(Deep)를 의미 있게 만드는 수학적 열쇠

① 퍼셉트론의 내부 연산(w1x1 + w2x2 + b)은 본질적으로 공간에 곧은 선을 긋는 선형 연산(직선)이다.

② 활성화 함수 없이 층만 깊게 쌓으면, 아무리 수백 층을 쌓아도 결국 하나의 큰 선형 함수(직선)가 된다.

y = w(wx+b)+b → y = w^2x + wb + b → y = Wx + B

③ 퍼셉트론이라는 '직선 자'를 유연하게 '구부리는 도구'가 바로 활성화 함수이다.

2) 복잡한 비선형 데이터(이미지, 텍스트)의 완벽한 처리(분류)

① 현실 세계의 복잡한 데이터(이미지, 텍스트 등)를 컴퓨터가 이해하도록 숫자 배열(임베딩)로 변환해 공간에 표현하면, 결코 직선으로 나눌 수 없는 구불구불하게 꼬인 비선형적 분포를 가진다.

② 퍼셉트론이 계산한 직선 신호를 활성화 함수로 구부리고 꺾어 줌으로써, 실타래처럼 꼬인 현실 데이터의 틈새를 정교하게 파고드는 '마법의 곡면 경계선'을 만들어내어 완벽한 분류(Classification) 문제를 풀어낼 수 있게 된다.

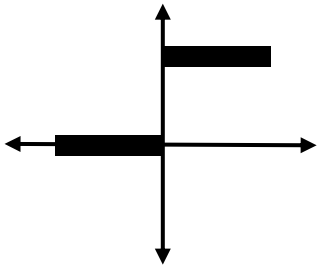
활성화 함수: 계단 함수 (Step Function)

- 정의 및 수학적 표현

1) 입력값이 0을 넘으면 1을 출력하고, 0 이하이면 0을 출력하는 가장 기본적인 활성화 함수이다.

2) 전통적인 단층 퍼셉트론으로 AND/OR/NAND 같은 논리 게이트 문제를 풀 때 사용한 이분법적 방식이다.

$$h(x) = \begin{cases} 0 & (x \leq 0) \\ 1 & (x > 0) \end{cases}$$



활성화 함수: 계단 함수 (Step Function)

- 한계점: 딥러닝 적용 불가

1) 이분법적 분류의 한계 (불연속성): 출력이 오직 0과 1이라는 극단적인 두 값으로만 잘리기 때문에, 맞췄는지 틀렸는지의 '모 아니면 도' 방식으로, 중간의 미세한 오차 변화를 반영할 수 없다.

2) 미분 불가능 (Gradient = 0): 0인 지점에서는 툭 끊어져 미분이 불가능하고, 0이 아닌 모든 구간에서는 그래프가 평평하여 미분값(기울기)이 완전히 0이 된다.

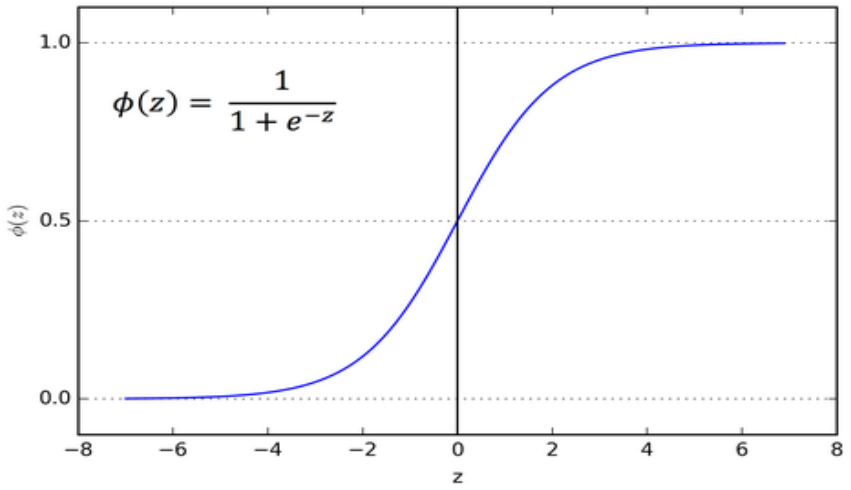
3) 오차 수정의 불가능: 현대 딥러닝은 발생한 오차를 기반으로 기울기를 구해 스스로 기울기(w)와 편차(b)를 수정하는 '경사하강법'을 사용하는데, 기울기가 0이 되면 컴퓨터가 오차를 바탕으로 학습을 진행할 방법이 전혀 없다.

활성화 함수: 시그모이드 함수 (Sigmoid Function)

- 정의 및 수학적 표현

1) 계단 함수의 거친 불연속성을 보완하여, 입력에 따라 출력이 S자 곡선 형태로 부드럽게 변하는 함수이다.

2) 초기 인공신경망의 전성기를 이끌었던 대표적인 비선형 활성화 함수이다.



활성화 함수: 시그모이드 함수 (Sigmoid Function)

- 주요 특징과 의의

  1) 연속적인 '정답 확률' 표현: 출력이 0과 1 사이의 실수로 떨어집니다. 이 덕분에 뉴런의 판단 결과를 "해당 정답 클래스일 확률(Probability)"로 해석할 수 있는 유연성을 얻었다.

  2) 자동 오차 수정(학습)의 시작: 모든 구간에서 부드러운 곡선이므로 '매끄러운 미분'이 가능하다. 출력이 나타내는 확률을 바탕으로 컴퓨터가 매개변수(w, b)를 스스로 최적화해 나가는 현대적 딥러닝 학습(오차역전파)의 기틀을 마련했다.

- 현대 거대 딥러닝에서의 한계점

  1) 포화(Saturation) 현상과 기울기 소실(Vanishing Gradient)

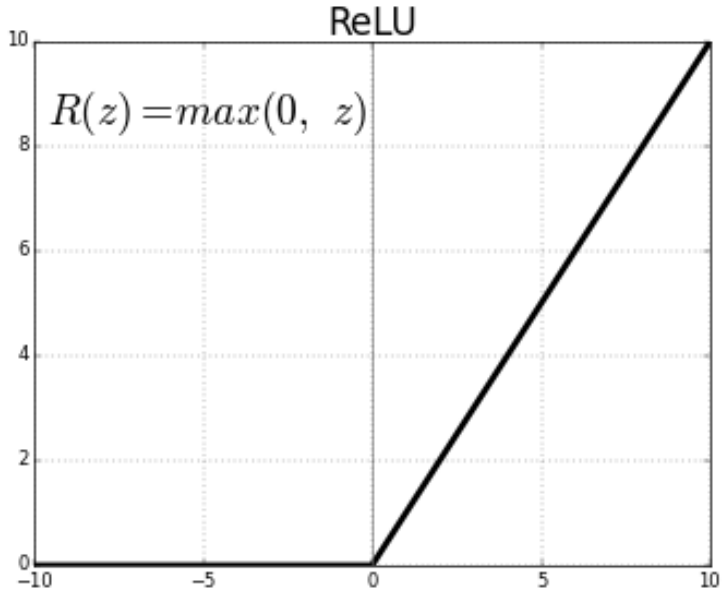
  2) 입력값 x가 너무 크거나 작아지면 곡선이 너무 평평해져서 시그모이드의 기울기(미분값)가 다시 0에 수렴하게 된다.

  3) 이로 인해 층이 수십, 수백 층으로 깊어지면 역전파 과정에서 오차를 수정할 기울기가 앞으로 전달되지 못하고 도중에 사라져, 딥러닝의 학습이 완전히 멈춰버리는 치명적인 문제를 야기한다.

활성화 함수: ReLU 함수

- 정의 및 수학적 표현

  : 입력이 0을 넘으면 입력값을 그대로 통과시키고, 0 이하이면 0으로 뚝 끊어 차단하는 함수



활성화 함수: ReLU 함수

- 현대 딥러닝에서 ReLU가 사실상의 표준인 이유

1) 경사 소실 문제의 완벽한 해결 (양수 영역)

① 입력값이 양수(x > 0)이기만 하면 그래프의 기울기가 항상 정확히 1로 유지된다.

② 이 성질 덕분에 ChatGPT와 같이 신경망의 층을 수백 층 깊게 쌓아 올려도 오차를 수정할 기울기가 죽지 않고 원형 그대로 초기 레이어 노드까지 완벽하게 전달된다.

2) 압도적인 연산 속도

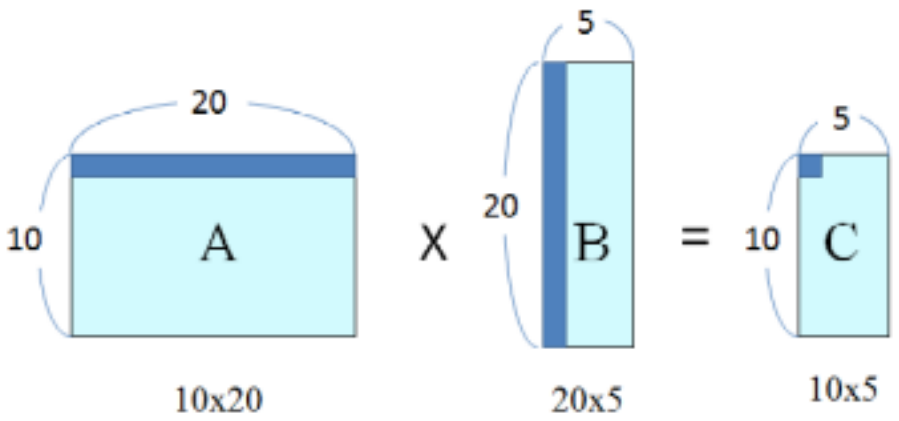
: 시그모이드처럼 복잡한 지수 연산을 할 필요 없이, 단순히 0보다 큰지 작은 지만 비교하면 되므로 모델의 연산 속도가 빨라진다.

활성화 함수 실습

다차원 배열의 계산

- 행렬 곱 / 벡터 내적

$$\underset{A}{\begin{pmatrix}1 & 0\\ 2 & 3\end{pmatrix}} \times \underset{B}{\begin{pmatrix}2 & 1\\ 3 & 1\end{pmatrix}} \rightarrow \begin{array}{|c|c|}\hline \begin{pmatrix}1 & 0\\ 2 & 3\end{pmatrix} \times \begin{pmatrix}2 & 1\\ 3 & 1\end{pmatrix} & \begin{pmatrix}1 & 0\\ 2 & 3\end{pmatrix} \times \begin{pmatrix}2 & 1\\ 3 & 1\end{pmatrix} \\ \hline \begin{pmatrix}1 & 0\\ 2 & 3\end{pmatrix} \times \begin{pmatrix}2 & 1\\ 3 & 1\end{pmatrix} & \begin{pmatrix}1 & 0\\ 2 & 3\end{pmatrix} \times \begin{pmatrix}2 & 1\\ 3 & 1\end{pmatrix} \\ \hline \end{array} \rightarrow \begin{bmatrix}(1\times2)+(0\times3) & (1\times1)+(0\times1)\\ (2\times2)+(3\times3) & (2\times1)+(3\times1)\end{bmatrix} = \begin{pmatrix}2 & 1\\ 13 & 5\end{pmatrix}$$



▶ 일반화

(m, k) X (k, n) = (m, n)

신경망(Neural Network) Layer와 행렬 내적

- 신경망 계산을 행렬로 표현해야 하는 이유

  1) 퍼셉트론 세포 하나는 개별 수식(y = w1x1 + w2x2 + b)으로 계산할 수 있지만, 많은 수의 뉴런이 얽힌 딥러닝에서는 연산 비효율성이 극대화됨

  2) 입력값들과 가중치들을 각각 하나의 다차원 배열(행렬)로 묶으면, 단 한 번의 행렬 곱셈 연산으로 층(Layer) 전체의 계산을 끝낼 수 있음

- 신경망의 연산 공식

  : 입력 벡터 X, 가중치 행렬 W, 편향 벡터 B가 주어졌을 때의 출력

$$Y = X \cdot W + B$$

$$\underset{(1,2)}{X} \times \underset{(2,3)}{W} + \underset{(3,)}{B} = \underset{(1,3)}{Y}$$

다차원 배열의 계산 실습

출력층(Output Layer)의 역할

- 인공신경망의 가장 마지막 단계로, 앞선 은닉층들을 거쳐 구부러지고 정제되며 흘러온 신호들을 최종 종합하여 우리가 원하는 예측 결과물을 생성해 내는 곳이다.

- 딥러닝의 문제 유형에 따라 출력층에 적용해야 하는 활성화 함수와 노드의 구조가 완전히 달라진다.

문제 유형에 따른 출력층 매핑 규칙

- 회귀 (Regression) -> 항등 함수 (Identity Function) 사용
  : 연속적인 수치(예: 주택 가격, 내일의 기온, 자율주행 조향 각도 등)를 왜곡 없이 그대로 맞히는 문제

- 이진 분류 (Binary Classification) → 시그모이드 함수 (Sigmoid Function) 사용
  : 둘 중 하나(예: 긍정 또는 부정)를 판별하여 '정답일 확률'을 도출하는 문제

- 다중 클래스 분류 (Multi-class Classification) → 소프트맥스 함수 (Softmax Function) 사용
  : 세 개 이상의 후보군(예: 개/고양이/새 분류) 중 가장 정답 같은 대상을 맞히는 문제

항등 함수 (Identity Function)

- 정의 및 수학적 표현

1) 입력된 신호를 아무런 변형이나 필터링 없이 그대로 다음 단계로 출력하는 함수입니다.

2) 수식적으로는 출력이 입력과 완벽히 일치하는 일차 직선의 형태를 띕니다.

$$f(x) = x$$

- 딥러닝에서의 의의

1) 72,000원이라는 가격이나 15.4도라는 제어 각도처럼, 수치 범위를 특정 확률(0~1)로 제한하거나 왜곡할 필요가 없는 회귀 문제의 특성을 반영한다.

2) 따라서 은닉층에서 최종 계산되어 나온 결과 행렬의 값(a)을 그대로 최종 결과(y)로 사용한다.

시그모이드 함수 (Sigmoid Function)

- 정의 및 수학적 표현

  : 앞선 은닉층에서 넘어온 최종 결과(x)를 0과 1 사이의 S자 곡선 값으로 압축하는 함수이다.

$$h(x) = \frac{1}{1 + e^{-x}}$$

- 딥러닝에서의 의의

  1) 출력이 이분법적인 0 또는 1로 단절되지 않고 0.85, 0.12 같은 연속적인 실숫값으로 생성된다.

  2) 덕분에 결과 값을 "이 데이터가 긍정일 확률이 85%이다"와 같이 확률적 신뢰도로 해석할 수 있게 해주며, 오차를 기반으로 역전파(학습)를 수행할 수 있는 이진 분류 출력 노드가 완성된다.

다중 클래스 분류의 표준: 소프트맥스 함수

- 정의 및 수학적 표현

: n개의 여러 출력 노드가 있을 때, 각 노드가 받은 입력 신호의 지수 함수를 전체 노드 지수 함수의 총합으로 나누어 전체적인 비율을 정규화하는 함수이다.

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^{n} \exp(a_i)}$$

- 소프트맥스 함수의 2대 핵심 특성

1) 출력값 범위의 정규화: 모든 출력층 뉴런의 출력값은 아무리 원본 점수가 크거나 작아도 항상 0과 1 사이의 실수 범주 안으로 안전하게 안착합니다 (0 < yk < 1).

2) 총합의 법칙: 소프트맥스 함수를 통과한 모든 출력 뉴런 값들을 다 더하면 반드시 '1(100%)'이 됩니다.

3) 의의: 총합이 1이 된다는 수학적 성질 덕분에, 딥러닝 모델의 복잡한 출력값들을 하나의 "확률 분포"로 완벽하게 대조하고 해석할 수 있다.

출력층의 뉴런 수 정하기 (Design Rule)

- 정답 알고리즘 설계 규칙

  : 출력층의 뉴런(노드) 개수는 기본적으로 내가 풀고자 하는 문제의 클래스(분류할 정답 대상)의 개수와 일치시켜야 한다.

- 대표적인 아키텍처 매핑 예시

  1) 이진 분류 (스팸 메일 판별)

  : '스팸일 확률'이라는 최종 수치 딱 1개만 도출 → 출력층 뉴런 수: 1개 (시그모이드 함수 연결)

  2) 다중 분류(MNIST 손글씨 숫자 데이터셋)

  : 맞혀야 하는 숫자가 0부터 9까지 총 10가지 종류 → 출력층 뉴런 수: 10개 (소프트맥스 함수 연결)

  3) 주택 가격 예측 (회귀)

  : 최종 정답 가격이라는 연속된 수치 1개 도출 → 출력층 뉴런 수: 1개 (항등 함수 연결 또는 없음)

출력층 설계하기 실습

손실 함수의 개념과 학습의 목적

- 손실(Loss)의 본질
  1) 오차의 크기: 딥러닝 모델이 예측한 값(y)과 실제 데이터가 가진 정답 레이블(t) 간의 거리이자 차이의 크기이다.
  2) 발생 원인: 신경망 내부의 핵심 매개변수인 가중치(W)와 편향(b)이 아직 주어진 데이터에 완벽히 최적화되지 않았기 때문에 발생한다.
- 손실 함수(Loss Function)의 정의와 역할
  1) 나쁨의 지표: 모델의 현재 성능이 얼마나 '나쁜가'를 정량적으로 나타내는 지표이다. 복잡한 수많은 오차의 크기를 단 하나의 명확한 스칼라(실수) 값으로 계산해 낸다.
  2) 딥러닝의 궁극적 목적: 이 손실 함수의 출력값을 최소화(0에 가깝게) 만드는 최적의 가중치 매개변수 조합을 찾아내는 과정이며, 이것이 바로 딥러닝이 말하는 '학습(Learning)'의 본질이다.

회귀 문제의 손실 함수: 평균 제곱 오차 (Mean Squared Error, MSE)

- 정의 및 수학적 표현

: 모델의 예측값과 실제 정답의 차이를 제곱하여 모두 더한 뒤 평균을 내는 함수이다. 주로 연속적인 수치(예: 주택 가격, 기온, 주가 등)를 예측하는 회귀(Regression) 문제에서 표준으로 사용된다.

$$E = \frac{1}{n}\sum_{i=1}^{n}(y_i - t_i)^2$$

- '제곱' 연산이 가지는 의미

1) 부호의 제거: 오차가 양수(+)든 음수(-)든 상관없이 제곱을 통해 모두 양수화하여, 순수한 오차의 절대적인 크기만을 반영한다.

2) 큰 오차에 대한 강력한 페널티: 오차가 크면 클수록 제곱 연산에 의해 손실 값이 기하급수적으로 커진다. 따라서 모델이 정답에서 크게 벗어난 치명적인 에러를 내지 않도록 강력하게 제어하는 효과를 준다

분류 문제의 손실 함수: 교차 엔트로피 오차 (Cross-Entropy Error)

- 정의 및 수학적 표현

1) 정답 레이블의 데이터 분포와 모델이 최종 출력층에서 예측한 확률 분포 사이의 수학적 거리를 계산하는 함수이다. 데이터가 어떤 카테고리에 속하는지 맞히는 분류(Classification) 문제에서 표준으로 사용된다.

$$E = -\sum_{k} t_k \log(y_k)$$

2) yk: 출력층의 소프트맥스 함수를 통과해 나온 모델의 예측 확률 값이다 (0 < yk < 1).

3) tk: 원-핫 인코딩(One-Hot Encoding) 형태로 변환된 실제 정답 레이블이다 (정답만 1, 오답은 모두 0)

분류 문제 유형별 교차 엔트로피 수식 구조

- 이진 분류 (Binary Classification)

1) 정의: 출력이 오직 0 또는 1이라는 두 가지 선택지로만 존재하는 경우 (예: 스팸 메일 여부 판별)

2) 수식

```
H(y, p) = - y * log(p) - (1 - y) * log(1 - p)
```

① H(y, p) : 손실 값

② y : 실제 정답 레이블 (0 또는 1)

③ p : 예측값 (시그모이드 함수를 통해 0과 1 사이의 실수값으로 변환된 값)

3) 손실 값 계산 예시

① 정답 레이블이 1이고, 모델이 예측한 값이 0.8이라고 가정

② y = 1, p = 0.8

③ 손실 값 : H(1, 0.8) = - 1 * log(0.8) - (1 - 1) * log(1 - 0.8)

= -log(0.8)

분류 문제 유형별 교차 엔트로피 수식 구조

- 다중 분류 (Multi-class Classification)

1) 정의: 분류해야 할 카테고리가 3개 이상인 경우 (예: 손글씨 숫자 0~9 분류)

2) 수식: 표준 교차 엔트로피 수식을 그대로 따른다

$$E = -\sum_{k} t_k \log(y_k)$$

3) 예시: 3개의 클래스(개, 고양이, 호랑이)를 분류하는 모델이 있다고 가정

① 1단계: 데이터 정의

- 실제 정답: 고양이 → t = [0, 1, 0] (원-핫 인코딩)
- 모델의 예측 확률: y = [0.1, 0.7, 0.2] (소프트맥스 출력 결과)

② 2단계: 공식 대입

E = - (0×log 0.1+1 × log 0.7+0 × log 0.2)

= -log 0.7

≒ 0.357

데이터 규모에 따른 학습 방식: 배치 vs 미니배치

- 손실 함수를 통해 오차를 구한 뒤 모델의 매개변수(가중치와 편향)를 업데이트할 때, "한 번에 얼마나 많은 데이터를 보고 손실을 계산할 것인가?"에 따라 크게 두 가지 학습 방식으로 나뉜다.
- 배치 학습 방식 (Batch Learning)
  1) 개념: 컴퓨터 메모리가 허용하는 한, 가지고 있는 전체 데이터 N개의 손실 값을 몽땅 구한 뒤, 이들의 평균 손실을 내어 매개변수를 딱 1번 업데이트하는 방식
  2) 특징: 전체 데이터를 모두 반영하므로 데이터가 수백만 개에 달하는 빅데이터 환경에서는 컴퓨터 메모리가 버티지 못하고 터지거나, 한 번 업데이트하는 데 시간이 너무 오래 걸린다는 치명적인 한계가 있음

데이터 규모에 따른 학습 방식: 배치 vs 미니배치

- 미니배치 학습 방식 (Mini-batch Learning)

  1) 개념: 전체 데이터 N개 중 임의의 작은 묶음인 n개(배치 크기)를 무작위로 추출하여, 그 n개 데이터 손실의 '평균'을 구한 뒤 매개변수를 기민하게 업데이트한다.

  2) 작동 프로세스:

  ① 전체 데이터 수를 N, 무작위 추출할 미니배치 크기를 n이라고 하면, 컴퓨터는 N/n번에 해당하는 횟수만큼 미니배치 묶음을 연속적으로 꺼내어 평균 손실을 구하고 매개변수를 그때마다 계속 업데이트한다.

  ② 마지막에 딱 맞아떨어지지 않고 남은 데이터 묶음이 있다면, 그 남은 개수만큼의 평균 손실을 구해 최종 업데이트를 진행하며 비로소 전체 데이터에 대해 1회 학습을 완료한 상태(1 에포크)가 된다.

데이터 규모에 따른 학습 방식: 배치 vs 미니배치

- 미니배치 학습 방식 (Mini-batch Learning)

  1) 개념: 전체 데이터 N개 중 임의의 작은 묶음인 n개(배치 크기)를 무작위로 추출하여, 그 n개 데이터 손실의 '평균'을 구한 뒤 매개변수를 기민하게 업데이트한다.

  2) 작동 프로세스:

  ① 전체 데이터 수를 N, 무작위 추출할 미니배치 크기를 n이라고 하면, 컴퓨터는 N/n번에 해당하는 횟수만큼 미니배치 묶음을 연속적으로 꺼내어 평균 손실을 구하고 매개변수를 그때마다 계속 업데이트한다.

  ② 마지막에 딱 맞아떨어지지 않고 남은 데이터 묶음이 있다면, 그 남은 개수만큼의 평균 손실을 구해 최종 업데이트를 진행하며 비로소 전체 데이터에 대해 1회 학습을 완료한 상태(1 에포크)가 된다.

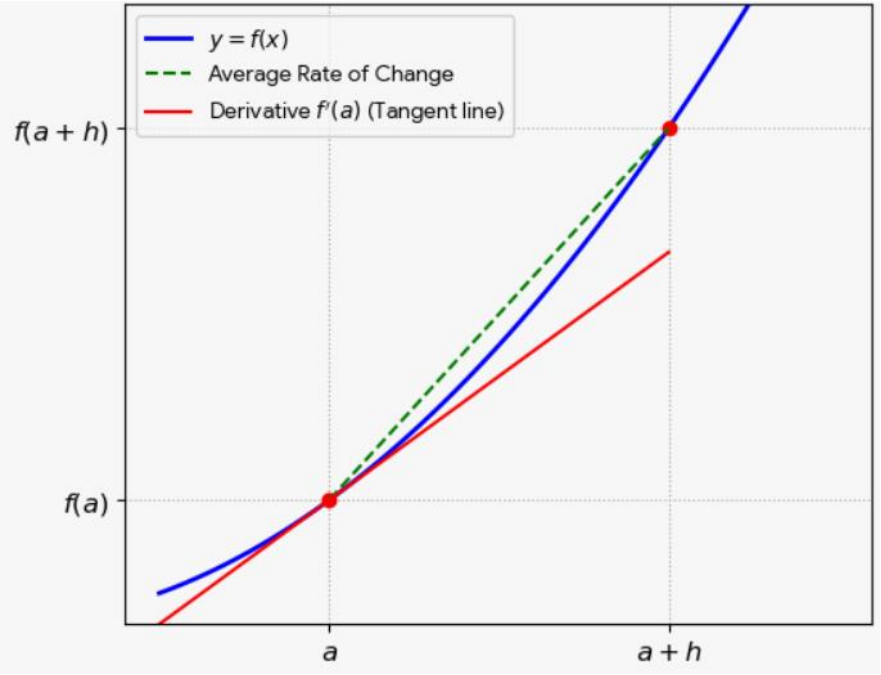
손실 함수 실습

미분의 개념과 딥러닝에서의 필요성

- 미분의 정의

1) 독립변수의 미세한 변화에 대해 함수 값이 그 지점에서 얼마나 민감하게 변하는지를 나타내는 '순간 변화율'

2) 기하학적으로는 함수 그래프 위 한 점에서의 '접선의 기울기'를 의미함



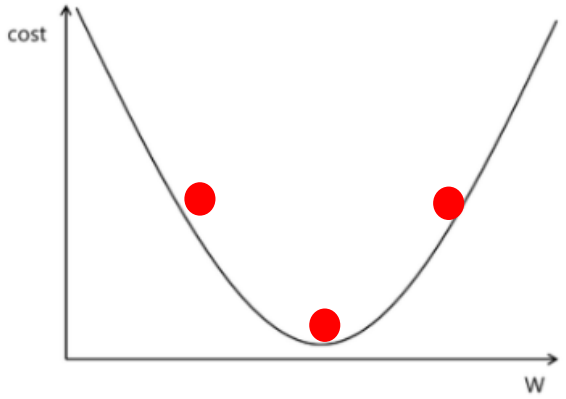
미분의 개념과 딥러닝에서의 필요성

- 딥러닝에서 미분이 반드시 필요한 이유

1) 우리는 앞 단원에서 정답과 모델 예측의 오차를 계산하는 손실 함수(Loss Function)를 배웠음

2) 우리의 최종 목표는 손실 함수(Loss)의 값을 0에 가깝게 줄이는 최적의 가중치(W)를 찾는 것임

3) 이때, "특정 가중치(W)를 아주 조금 바꿨을 때, 손실 함수의 값이 늘어나는가? 줄어드는가?"를 판별하는 유일한 도구가 바로 미분(기울기)임



4) 미분 값(기울기)이 양수(+)이면 -> 가중치를 (-) 방향으로 조절

5) 미분 값(기울기)이 음수(-)이면 -> 가중치를 (+) 방향으로 조절

수치 미분(Numerical Differentiation)

- 해석적 미분 vs 수치 미분

1) 해석적 미분: 우리가 고등학교 수학 시간에 배운 수학 공식을 사용($x^2 \rightarrow 2x$),
오차가 없는 정확한 값을 구할 수 있지만 복잡한 딥러닝 수식을 컴퓨터가 매번 자동으로 공식을 적용하기에는 한계가 있음

2) 수치 미분: 아주 미세한 차이(h)를 대입하여 수치적인 근사치로 기울기를 구하는 방식. 컴퓨터가 단순 반복 연산으로 고속 처리하기에 가장 최적화되어 있음

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

수치 미분(Numerical Differentiation)

- 미분의 예시

1) 간단한 2차 함수 미분

① 함수 : f(x) = x^2

② 목표 : 특정 지점 x = 3에서의 미분값(기울기) 구하기

2) 해석적 미분 결과

① 공식 적용 : f'(x) = 2x

② x = 3 대입 : 2 * 3 = 6 (오차 없는 절대 정답)

3) 컴퓨터 방식(수치 미분) 결과 (h = 0.0001 차분 적용)

$$f(x+h) = f(3.0001) = 3.0001^2 = 9.00060001$$

$$f(x-h) = f(2.9999) = 2.9999^2 = 8.99940001$$

공식 대입 :

$$\frac{9.00060001 - 8.99940001}{2 \times 0.0001} = \frac{0.0012}{0.0002} = \mathbf{6.0000}$$

4) 결론 : 복잡한 미분 공식을 컴퓨터에게 가르치지 않아도, 수치 미분 공식을 활용하면 소수점 아래까지 완벽히 일치하는 기울기를 고속으로 계산해낼 수 있음

다변수 함수를 위한: 편미분 (Partial Derivative)

- 편미분의 정의
  1) 입력 변수가 여러 개인 함수(다변수 함수)에서, "내가 관심 있는 단 하나의 변수만 미분하고, 나머지 변수는 모두 고정된 상수(숫자)로 취급"하는 미분 방식
  2) 기호로는 d 대신 ∂(라운드)를 사용함 : $\frac{\partial f}{\partial x_0}$
- 딥러닝에서 편미분이 가지는 절대적 의의
  1) 인공신경망의 손실 함수는 변수가 단 하나가 아니라, 수천만 ~ 수천억 개의 가중치 매개변수(w1, w2, w3, ...)로 얽혀 있음
  2) 수많은 가중치 중 오직 w1 하나를 아주 미세하게 조정했을 때 손실 함수 오차가 어떻게 변하는가?를 알기 위해 편미분이 사용됨
  3) 각 가중치별로 편미분을 모두 구해 1차원 배열로 묶은 것을 기울기(Gradient)라고 함

다변수 함수를 위한: 편미분 (Partial Derivative)

- 편미분의 예시: 2개의 가중치가 얽힌 가상 손실 함수에 관한 문제

  1) 함수: $f(x_0, x_1) = x_0^2 + x_1^2$

  2) 목표 : (x_0= 3, x_1 = 4) 일 때 각각의 변수로 편미분 수행

  3) 문제와 풀이

[문제 1] x_0에 대한 편미분 ($\frac{\partial f}{\partial x_0}$)

- **규칙** : x_1은 아무 힘이 없는 단순한 숫자(상수 4)로 취급하여 미분 시 소멸시킴
- 수식의 단순화 : $f(x_0) = x_0^2 + 4^2 \rightarrow x_0^2 + 16$
- x_0로 미분 후 3 대입 : $2x_0 \rightarrow 2 \times 3 = \mathbf{6}$

[문제 2] x_1에 대한 편미분 ($\frac{\partial f}{\partial x_1}$)

- **규칙** : 이번에는 반대로 x_0를 단순한 숫자(상수 3)로 취급하여 소멸시킴
- 수식의 단순화 : $f(x_1) = 3^2 + x_1^2 \rightarrow 9 + x_1^2$
- x_1로 미분 후 4 대입 : $2x_1 \rightarrow 2 \times 4 = \mathbf{8}$

수치 미분 실습

경사하강법과 가중치 업데이트의 표준 수식

- 명확한 변수 정의

1) 종속 변수

① 손실 함수 (L, Loss Function)

② 모델의 예측 확률 분포와 실제 정답 레이블 분포 간의 오차 크기를 나타내는 단 하나의 실수 값

2) 독립 변수

① 가중치 행렬 (W, Weights), 편향(b)

② 신경망의 은닉층과 출력층 내부에서 입력 신호의 영향력을 조절하며, 궁극적으로 손실 함수의 출력값(L)을 결정짓는 조절 가능한 독립 변수 매개변수들

경사하강법과 가중치 업데이트의 표준 수식

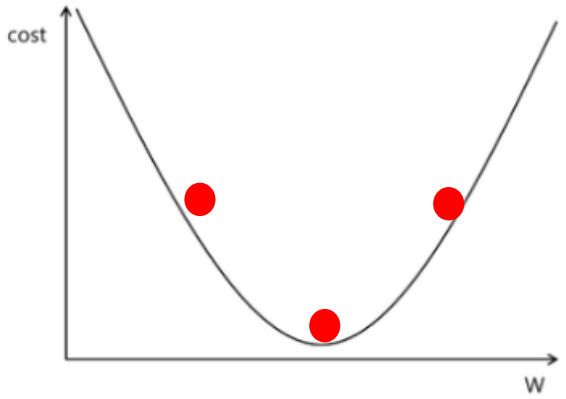
- 가중치 매개변수 업데이트 공식

1) 손실 함수(L)를 최소화하기 위해 독립 변수인 각 층의 가중치 행렬들을 기울기의 정반대 방향으로 수정함

$$W_{1new} = W_{1old} - \eta \frac{\partial L}{\partial W_1}, \quad W_{2new} = W_{2old} - \eta \frac{\partial L}{\partial W_2}$$

2) $\frac{\partial L}{\partial W_1}$, $\frac{\partial L}{\partial W_2}$: 각 독립 변수 가중치 행렬이 손실 함수 L에 미치는 순간 변화율인 기울기 배열(행렬)

3) 학습률 (Learning Rate) eta: 독립 변수를 한 걸음에 얼마나 수정할지 결정하는 보폭



다층 신경망 아키텍처에서의 행렬 모양(Shape) 예시

- 미니배치 입력 데이터 (X)

  : 미니배치 크기 n=4, 입력 특성 수 d=2 일 때, 입력 행렬의 모양 : (4, 2)

- 은닉층(Hidden Layer) 연산 구체화

  1) 단 1개의 노드(퍼셉트론)은 비선형 분리가 불가능함. 다각도의 특징 병렬 학습을 위해 다수의 은닉층 노드가 필요함(은닉층 노드 m=3으로 가정)

  2) 은닉층 가중치 행렬 (W1) : 은닉층 노드 수가 3개 이므로 행 (2, 3)의 모양이어야 함

  3) 은닉층 출력 행렬 (Z): X • W1 연산 후 모양은 (4, 3)이 됨, 비선형 활성화 함수(ReLU)를 통과

- 출력층(Output Layer) 연산 구체화

  1) 다중 분류 예측 : 예측할 정답 클래스의 개수가 k=3개인 다중 분류 과제 → 출력층 노드 3개

  2) 출력층 가중치 행렬 (W2) : 은닉층 출력 Z를 받아서 출력층 노드 3개를 연결해야 하므로 (3, 3)의 모양이어야 함

  3) 최종 출력 행렬 (Y): Z • W2 연산 후 소프트맥스(Softmax)를 통과하여 도출되는 4개 미니배치 데이터에 대한 3개 클래스별 예측 확률 행렬, 모양은 (4, 3)이 됨

정답 레이블(T) 구조와 기울기 행렬의 차원

- 다중 분류 정답 레이블 행렬 (T)의 구조
  1) 다중 분류에서 미니배치 4개의 정답은 각 데이터가 어떤 클래스(3개 중 하나)에 속하는지 명시하는 원-핫 인코딩(One-Hot Encoding) 형태로 구조화되어야 함([1 0 0], [0 1 0], [0 0 1])
  2) 따라서 정답 레이블 행렬 T의 모양은 4개 데이터의 3개 클래스 분포를 표현하는 (4, 3) 형태의 행렬이 됨
- 모양 일치의 법칙 (Shape Match Rule)에 의한 기울기 차원 결정
  1) 최종 손실 함수 L은 4개 데이터의 교차 엔트로피 오차를 평균 낸 단 하나의 실수임
  2) 가중치 행렬 W1과 W2의 각 원소들이 손실 L에 미치는 편미분 값을 계산할 때, 연산 행렬의 차원 보존 법칙에 의해 기울기 행렬의 형상은 대응하는 독립 변수(가중치)의 모양과 완벽하게 일치해야 함
  3) 결론
     ① 은닉층 가중치 W1의 형상이 (2, 3)이므로, 기울기 행렬 $\frac{\partial L}{\partial W_1}$ 의 모양도 반드시 (2, 3) 2차원 행렬임
     ② 출력층 가중치 W2의 형상이 (3, 3)이므로, 기울기 행렬 $\frac{\partial L}{\partial W_2}$ 의 모양도 반드시 (3, 3) 2차원 행렬임

기울기(Gradient) 실습

복잡함을 단순하게: 계산 그래프와 국소적 계산

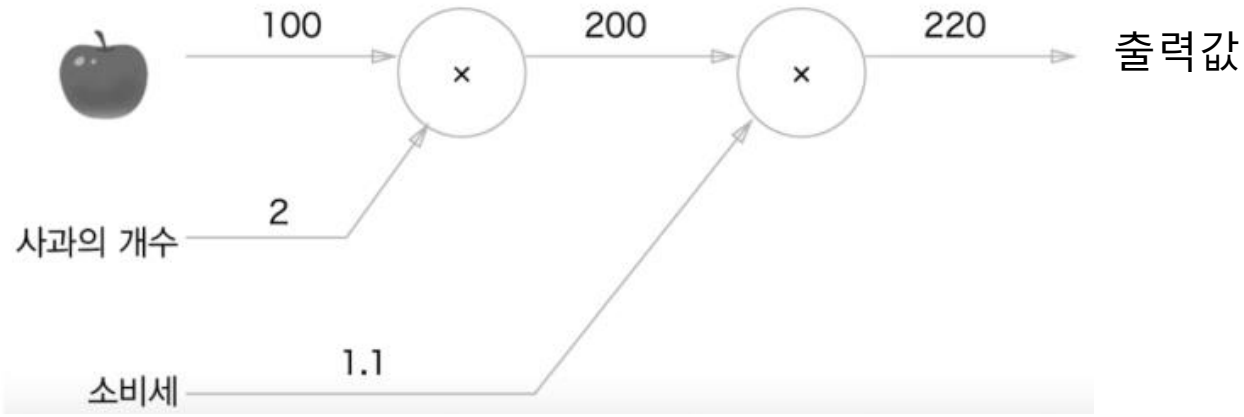
- 계산 그래프(Computational Graph)의 정의

1) 복잡한 수학의 계산 과정을 '노드(Node, 연산 기호)'와 '에지(Edge, 데이터가 흐르는 선)'로 이루어진 네트워크 그래프로 시각화하여 표현한 것

2) 순방향 연산 (Forward Pass) : 입력층에서 시작하여 출력층 방향으로 순서대로 연산을 진행하여 최종 결과(예측값 또는 손실값)를 구해내는 과정

3) 역방향 연산 (Backward Pass) : 출력층에서 시작하여 입력층 방향(반대 방향)으로 거꾸로 이동하며 기울기(Gradient)를 계산하는 과정

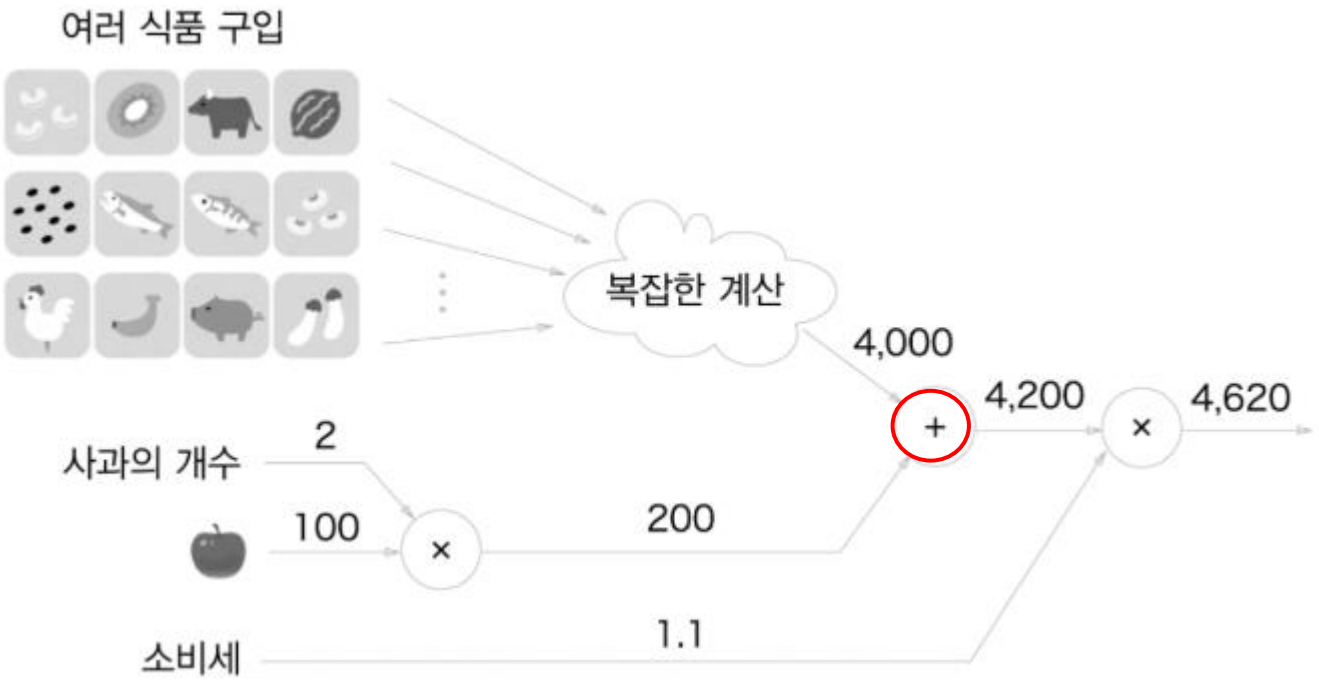
문제: 현빈 군은 슈퍼에서 1개에 100원인 사과를 2개 샀습니다.
이때 지불 금액을 구하세요.
단, 소비세가 10% 부과됩니다



복잡함을 단순하게: 계산 그래프와 국소적 계산

- 국소적 계산 (Local Computation)

  : 나만 신경 쓴다 : 전체 신경망이 아무리 거대하고 수천만 개의 연산이 얽혀 있어도, 개별 노드는 "자신에게 입력되는 데이터"와 "자신이 출력하는 데이터"라는 아주 좁은 범위(국소적)의 연산만 처리하는 계산 방식



'더하기(+)'라는 연산 노드는 전체 과정을 전혀 알 필요가 없습니다.
오직 자신에게 입력된 두 숫자인 4,000과 200만을 가지고 4,200이라는 결과를 내보냅니다

새로운 개념: 합성함수(Composite Function)의 이해

- 합성함수(Composite Function)란 무엇인가?
  1) 여러 개의 함수가 사슬처럼 줄지어 엮여서, 한 함수의 출력이 다음 함수의 입력으로 이어지는 구조의 함수
  2) 딥러닝 모델의 실체가 바로 합성함수임 (입력층 → 은닉층 함수 → 출력층 함수 → 손실 함수)
- 일상 속의 예시 (마트에서 사과 사기)
  1) 사과의 원가(x)가 변하면, 과일 가게의 마진이 붙은 최종 사과 가격(u)이 변하고, 거기에 10% 소비세가 붙어 소비자가 내는 최종 금액(y)이 결정됨
  2) 수식 표기 : u = g(x), y = f(u)
  3) 최종 합성함수 : y = f(g(x))
  4) 질문: "사과 원가(x)가 1원 변할 때, 최종 소비자가 내는 금액(y)은 얼마나 변할까?"

     ⇒ 전체 금액에 대한 원가 x의 미분 값: $\frac{\partial y}{\partial x}$

연쇄법칙 (Chain Rule)

- 연쇄법칙(Chain Rule)의 정의

  : 합성함수의 미분은 합성함수를 구성하는 개별 함수의 미분(국소적 미분)들을 차례대로 모두 곱해서 구할 수 있다는 법칙

- '마트에서 사과 사기' 예시의 적용

  1) 최종 소비자 금액(y)을 사과 원가(x)로 한 번에 미분하는 공식을 몰라도, 단계를 쪼개면 쉬워짐

     ① 1단계 미분 : 사과 가격(u)이 변할 때 최종 금액(y)의 변화량

     ② 2단계 미분 : 사과 원가(x)가 변할 때 사과 가격(u)의 변화량

  2) 연쇄법칙에 의한 결합

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \times \frac{\partial u}{\partial x}$$

  3) 결과: 단계별 미분 결과를 곱해주었더니, 가운데 매개 변수인 u가 마치 분수처럼 약분되어 지워지면서 우리가 원하던 최종 기울기 $\frac{\partial y}{\partial x}$가 깔끔하게 도출됨

순방향 연산과 역방향 연산

- 순방향 연산 (Left to Right)
  1) 마트에서 계산이 일어날 때는 원가(x)가 입력되어 가격(u)이 되고, 세금이 붙어 최종 금액(y)이 결정됨
  2) 왼쪽(입력)에서 오른쪽(출력)으로 흘러감 → 순방향 연산 (Forward Pass)

$$\text{원가 } (x) \longrightarrow [\text{마진 노드}] \longrightarrow \text{가격 } (u) \longrightarrow [\text{소비세 노드}] \longrightarrow \text{최종 금액 } (y)$$

- 기울기의 계산과 역방향 연산 (Right to Left)
  1) 연쇄법칙 공식은 $\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \times \frac{\partial u}{\partial x}$ 임
  2) **공식의 의미: 맨 오른쪽 끝의 결과물인 $\frac{\partial y}{\partial u}$를 구한 뒤, 그 신호를 왼쪽으로 보내고 $\frac{\partial u}{\partial x}$ 와 곱하면 최종 결과에 도달할 수 있음**
  3) 이처럼 미분의 결과를 출력(오른쪽)에서 입력(왼쪽) 방향으로 거꾸로 전달하는 행위를 "역전파(Backpropagation, 거꾸로 전파됨)"라고 부름

사과 예시와 인공신경망의 1:1 대응

- 앞에서 살펴본 사과의 가격 결정 과정은 신경망의 계산 흐름과 정확히 같다.
  1) 사과 예시: 원가(x) → 판매 가격(u) → 최종 결제 금액(y)
  2) 인공신경망: 은닉층 퍼셉트론 입력 신호 총합($z = w \times x + b$) → 활성화 함수($a = \sigma(z)$) → … → 최종 손실값(L)
  3) 즉, 인공신경망의 본질은 수많은 퍼셉트론들이 사슬처럼 엮인 하나의 거대한 '합성함수 ' 이다.

우리가 찾아야 하는 최종 목표

- 퍼셉트론의 가중치 기울기(∂L/∂w)

1) 딥러닝 학습의 목표는 최종 오차인 손실 함수(Loss, L)를 0에 가깝게 줄이는 것이다.

2) 그러기 위해서는 모든 퍼셉트론의 가중치(w)를 최적의 가중치로 업데이트 해야 한다.

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial L}{\partial W}$$

3) 즉, 우리가 구하고자 하는 최종 계산값은 바로 특정 퍼셉트론의 '손실(L)에 대한 가중치(w)의 기울기(∂L/∂w) ' 이다.

연쇄법칙과 '오차역전파'로 은닉층 가중치 기울기(∂L/∂w) 도출하기

- 연쇄법칙(Chain Rule)을 통한 기울기 분해

  1) 은닉층 특정 퍼셉트론의 가중치 기울기(∂L/∂w)는 연쇄법칙에 의해 단 3개의 '국소적 미분의 곱'으로 깔끔하게 풀린다.

  2) 공식 : $\partial L/\partial w = (\partial L/\partial a) \times (\partial a/\partial z) \times (\partial z/\partial w)$

- 3단계 국소적 미분의 구체적 계산 (나만 신경 쓰면 되는 국소적 계산)

  1) 1단계: $\partial L/\partial a$

  • 내 뒤에 있는 출력층들이 이미 계산해서 거꾸로 넘겨준 오차 신호를 그대로 전달받아 사용

  ② 2단계: $\partial a/\partial z$

  • 해당 퍼셉트론의 활성화 함수(예: ReLU)에 대한 미분

  ③ 3단계: $\partial z/\partial w$

  • 퍼셉트론의 입력의 총합 연산($z = w \times x + b$)을 가중치 w로 미분한 결과 → 해당 퍼셉트론으로 들어온 '입력값(x)' 그 자체

연쇄법칙과 '오차역전파'로 은닉층 가중치 기울기(∂L/∂w) 도출하기

- 오차역전파(Backpropagation)의 본질과 의의
  1) 은닉층의 특정 퍼셉트론은 전체 신경망의 복잡한 (1,000만 개) 연산을 알 필요가 없다.
  2) "뒤에서 건너온 오차(∂L/∂a)"에 "자신의 활성화 함수에 대한 미분(∂a/∂z)"과 "입력값(x)"만 곱해주면 자신의 가중치 기울기(∂L/∂w)가 즉시 완성된다.
  3) 가중치마다 순방향 계산을 수없이 반복할 필요 없이, 출력층에서 딱 한 번 뒤로 오차를 보내는(역전파) 것만으로 모든 은닉층 가중치의 기울기를 일괄 산출할 수 있다.

최적화(Optimization)

- 최적화 : 손실 함수의 값(오차)이 최소가 되는 최적의 가중치를 찾는 과정
- 최적화 알고리즘
  1) 최적의 가중치를 찾아가는 구체적인 방법과 수단
  2) 기본 : 확률적 경사 하강법(SGD)

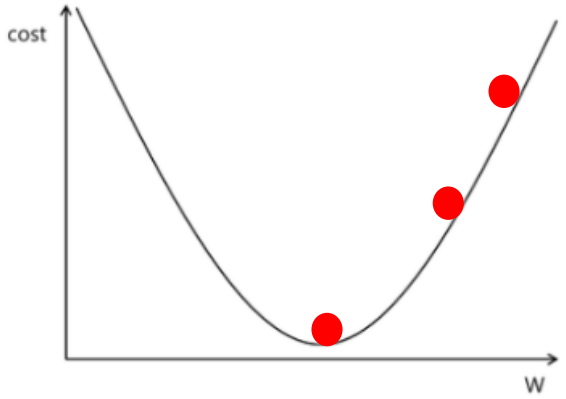
확률적 경사 하강법 (SGD)의 정의와 한계

- 공식

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial L}{\partial W}$$

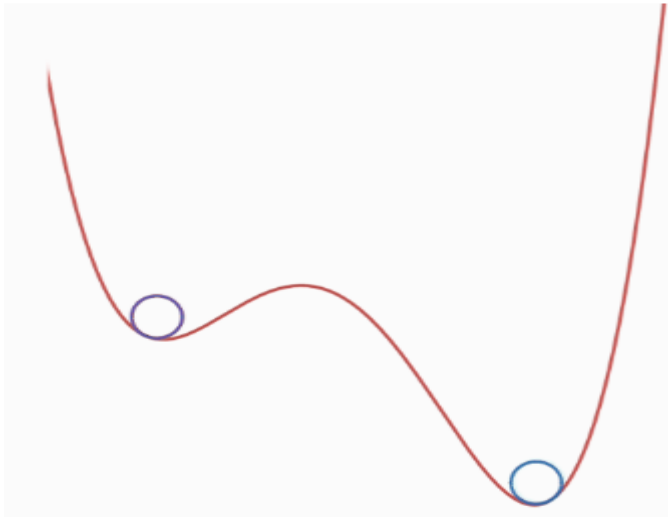
개선된 가중치 = 현재 가중치 – (learning_rate X 현재 가중치에서의 경사(기울기))

- 작동 방식: 현재 위치에서 기울기가 감소하는 내리막길 방향으로, 설정한 보폭(업데이트 된 가중치의 크기)만큼 정직하게 한 걸음씩 걸어 내려가는 방식



확률적 경사 하강법 (SGD)의 정의와 한계

- 치명적 한계 (로컬 미니멈 문제):
  1) 가짜 최소인 로컬 미니멈(지역 극소점) 바닥에 도달하면, 주변이 모두 오르막이라 순간적으로 가중치의 기울기가 0이 됨
  2) 공식에 의해 보폭이 0이 되므로 학습이 완전히 멈춰 버림. 결국 가운데 솟아 있는 언덕을 넘지 못하고, 반대편에 있는 진짜 목적지인 전역 최소점(Global Minimum)을 눈앞에 둔 채 가짜 골짜기에 영원히 갇히게 됨



모멘텀 (Momentum)

- 모멘텀(Momentum)의 의미

1) '운동량' 또는 '관성'을 뜻하는 물리학 용어로, 가중치가 이동할 때 과거에 이동했던 방향과 속도를 기억하여 가속도를 붙여주는 알고리즘

2) 일상 속의 비유

"경사면에서 굴러 내려오는 '무거운 쇠구슬'을 상상해 보십시오.
쇠구슬은 굴러내려 오던 거대한 탄력(가속도) 때문에, 가짜 골짜기(로컬 미니멈) 바닥에 도달해도
멈추지 않고 그 관성의 힘으로 앞의 언덕을 웅장하게 치고 올라가 넘어가게 됩니다."

모멘텀 (Momentum)

- 수식 정리

  : 속도 행렬 v를 도입하여 과거의 움직임을 누적 보존함 (보통 관성 계수 alpha=0.9 사용)

$$v_{\text{new}} = \alpha v_{\text{old}} - \eta \frac{\partial L}{\partial W}$$

$$W_{\text{new}} = W_{\text{old}} + v_{\text{new}}$$

- 효과

  1) 지역 극소점 바닥에서 현재 기울기가 0이 되더라도, 과거에서부터 누적된 속도가 강력하게 살아있음

  2) 이 탄력의 힘으로 가짜 골짜기를 탈출하여 가운데 언덕 장벽을 통과해 반대편 진짜 목적지로 전진함

AdaGrad

- 기본 원리

1) 모든 가중치들에게 똑같은 학습률을 고정으로 적용하는 것은 비효율적이다.

2) AdaGrad는 과거 학습 과정에서 업데이트가 많이 진행된 가중치의 경우, 학습률을 강하게 감소시켜 가중치 업데이트의 크기(보폭)을 촘촘하게 줄여준다.

3) 반면, 아직 학습이 거의 안 되어 있는 가중치이 경우, 초기 학습률을 최대한 유지하여 손실 함수 최소 지점에 빠르게 따라붙게 만든다.

4) 결과적으로 가중치의 학습(이동) 이력에 따라 보폭의 완급을 스스로 조율하는 가중치별 맞춤형 최적화 알고리즘이다

AdaGrad

- 수식 정리 (기울기 제곱 누적 행렬 h: 학습의 진행 정도를 표시)

  : 학습이 진행되는 동안 측정된 기울기들을 제곱하고 계속 누적하여 학습의 진행 정도를 표시할 '주행거리 계기판'에 해당하는 행렬 h를 도입함

$$h_{\text{new}} = h_{\text{old}} + \left(\frac{\partial L}{\partial W}\right) \odot \left(\frac{\partial L}{\partial W}\right)$$

$$W_{\text{new}} = W_{\text{old}} - \frac{\eta}{\sqrt{h_{\text{new}}} + \epsilon} \frac{\partial L}{\partial W}$$

- AdaGrad의 치명적 한계

  1) 가짜 골짜기(로컬 미니멈)까지 오는 동안 학습 시간이 오래 걸리면, 계기판 h 값이 분모 자리에서 무한히 커짐

  2) 목적지(전역 최소점) 근처가 아닌대도 단지 학습이 많이 진행이 되었다는 이유만으로 보폭이 완전히 0에 가깝게 되어 가짜 골짜기를 넘지 못하고 학습이 종료됨

현대 딥러닝의 표준: Adam

- 두 알고리즘의 완벽한 융합 (Momentum + AdaGrad)
  1) "쇠구슬의 묵직한 관성(Momentum)"과 "가중치별 보폭 자율 조정(AdaGrad)"의 장점만을 영리하게 한 바구니에 담은 최적화 알고리즘
  2) 딥러닝에서 가장 널리 사용되는 최적화 알고리즘
- AdaGrad의 단점을 완벽하게 극복
  1) 기억 상실 기능 도입: 과거의 기울기 주행거리 기록(과거에 누적된 기울기 제곱) 을 일정 비율로 계속 깎아내어(감쇠시켜) 자연스럽게 제거함
  2) 대신 최근에 생성되어 누적되는 최신 기울기 위주로 반영하기 때문에, 누적되는 기울기 제곱의 수치가 무한대로 폭발하지 않음.
  3) 따라서 로컬 미니멈처럼 경사가 완만한 지점에 도달해도 유효 학습률이 0이 되지 않음

Pytorch를 이용한 딥러닝 모델 개발

- nn.Linear(in_features, out_features)
  1) 은닉층(Hidden Layer)과 출력층(Output Layer)을 만드는 도구
  2) 매개 변수 설명
     ① in_features (입력 특성 수): 이전 단계에서 들어오는 데이터의 개수
     ② out_features (출력 뉴런 수): 현재 계층에 배치할 새로운 인공 뉴런(퍼셉트론)의 개수
     ③ 예시: nn.Linear(4, 16) → 이전 단계 입력 4개, 현재 계층의 인공 뉴런 16개

Pytorch를 이용한 딥러닝 모델 개발

- nn.Sequential()
  1) 여러 층을 순서대로 쌓아 하나로 묶어주는 도구
  2) 예시

```
binary_model = nn.Sequential(
    nn.Linear(4, 16),    # 은닉층 1: 입력 4개, 뉴런 16개
    nn.ReLU(),           # 활성화 함수, 비선형 연산 수행
    nn.Linear(16, 8),    # 은닉층 2: 입력 16개, 뉴런 8개
    nn.ReLU(),           # 활성화 함수, 비선형 연산 수행
    nn.Linear(8, 1)      # 은닉층 2 → 출력층: 최종 판단 노드 1개
)
```

Pytorch를 이용한 딥러닝 모델 개발

- 분류(Classification) 문제: 모델 설계, 학습, 평가

1) 분류 모델 정의 방법 (nn.Sequential)

① 이진 분류 모델

```
binary_model = nn.Sequential(
    nn.Linear(4, 16),     # 입력층 → 은닉층 1: 입력 4개, 뉴런 16개
    nn.ReLU(),            # 활성화 함수, 비선형 연산 수행
    nn.Linear(16, 8),     # 은닉층 2: 입력 16개, 뉴런 8개
    nn.ReLU(),            # 활성화 함수, 비선형 연산 수행
    nn.Linear(8, 1)       # 은닉층 2 → 출력층: 최종 판단 노드 1개
)
```

Pytorch를 이용한 딥러닝 모델 개발

② 다중 분류 모델

```
multi_model = nn.Sequential(
    nn.Linear(4, 16),     # 입력층 → 은닉층 1: 입력 4개, 뉴런 16개
    nn.ReLU(),            # 활성화 함수, 비선형 연산 수행
    nn.Linear(16, 8),     # 은닉층 2: 입력 16개, 뉴런 8개
    nn.ReLU(),            # 활성화 함수, 비선형 연산 수행
    nn.Linear(8, 3)       # 출력층: 최종 클래스 개수(3개)만큼 출력 노드 생성
)
```

Pytorch를 이용한 딥러닝 모델 개발

- 분류(Classification) 문제: 모델 설계, 학습, 평가

2) 손실 함수와 옵티마이저 정의

① 이진 분류 손실 함수: criterion = nn.BCEWithLogitsLoss()

- 시그모이드와 이진 교차 엔트로피(BCE)를 결합한 함수
- 정답 데이터(y): 반드시 (N, 1) 형상의 FloatTensor (0.0 또는 1.0)여야 한다.

② 다중 분류 손실 함수: criterion = nn.CrossEntropyLoss()

- 소프트맥스와 다중 교차 엔트로피(CEE)를 결합한 함수
- 정답 데이터(y): 원-핫 인코딩이 아닌 (N,) 형상의 정수형 LongTensor (0, 1, 2, ...)여야 한다.

③ 옵티마이저: optimizer = optim.Adam(model.parameters(), lr=0.01)

- 손실을 최소화하는 방향으로 모델 내 모든 가중치(model.parameters())를 자동 갱신

Pytorch를 이용한 딥러닝 모델 개발

- 분류(Classification) 문제: 모델 설계, 학습, 평가

3) 학습 루프 (Training Loop: 5대 핵심 공식)

```
model.train()  # 1. 모델을 학습 모드로 설정

for epoch in range(epochs):
    # ① 기울기 초기화: 이전 루프에서 누적된 기울기를 0으로 리셋 (필수!)
    optimizer.zero_grad()

    # ② 순전파(Forward Pass): 입력(X)을 모델에 넣어 예측 점수(Logits) 계산
    outputs = model(X_train)

    # ③ 손실 계산: 예측 점수와 실제 정답(y) 사이의 오차 산출
    loss = criterion(outputs, y_train)

    # ④ 역전파(Backpropagation): 손실을 가중치로 편미분하여 기울기(Gradient) 계산
    loss.backward()

    # ⑤ 매개변수 갱신(Step): 계산된 기울기 방향으로 가중치 수정
    optimizer.step()
```

Pytorch를 이용한 딥러닝 모델 개발

- 분류(Classification) 문제: 모델 설계, 학습, 평가

4) 모델 평가 및 성능 측정

```
model.eval()  # 1. 모델을 평가/추론 모드로 전환

with torch.no_grad():  # 2. 평가 시에는 미분 계산을 꺼서 메모리와 연산 절약
    test_logits = model(X_test)

    # [이진 분류 최종 라벨 판정]
    # Logits에 Sigmoid를 씌워 확률로 바꾼 뒤 0.5 이상이면 1, 미만이면 0
    probs = torch.sigmoid(test_logits)
    predictions = (probs >= 0.5).int().numpy()

    # [다중 분류 최종 라벨 판정]
    # 클래스 축(dim=1)을 기준으로 가장 점수(확률)가 높은 방 번호(인덱스) 선택
    # predictions = torch.argmax(test_logits, dim=1).numpy()

# 3. 사이킷런의 정확도(Accuracy) 지표로 최종 검증
acc = accuracy_score(y_test, predictions)
print(f"평가 정확도: {acc * 100:.2f}%")
```

Pytorch를 이용한 딥러닝 모델 개발

- 회귀(Regression) 문제: 모델 설계, 학습, 평가

1) 회귀 모델 정의 방법 (nn.Sequential)

```
regression_model = nn.Sequential(
    nn.Linear(3, 32),
    nn.ReLU(),
    nn.Linear(32, 16),
    nn.ReLU(),
    nn.Linear(16, 1)      # 출력층: 예측할 수치값이 1개이므로 출력 노드는 1개
)
```

Pytorch를 이용한 딥러닝 모델 개발

- 회귀(Regression) 문제: 모델 설계, 학습, 평가

2) 손실 함수와 옵티마이저 정의

① 회귀 손실 함수: criterion = nn.MSELoss() (평균 제곱 오차)

· 예측값과 실제값의 차이를 제곱하여 평균을 낸 지표이다.

② 옵티마이저: optimizer = optim.Adam(model.parameters(), lr=0.01)

· 손실을 최소화하는 방향으로 모델 내 모든 가중치(model.parameters())를 자동 갱신

Pytorch를 이용한 딥러닝 모델 개발

- 분류(Classification) 문제: 모델 설계, 학습, 평가

3) 학습 루프 (Training Loop: 5대 핵심 공식)

```
model.train()

for epoch in range(epochs):
    optimizer.zero_grad()                    # ① 기울기 리셋
    predictions = model(X_train)             # ② 순전파 (수치 예측)
    loss = criterion(predictions, y_train)   # ③ MSE 오차 계산
    loss.backward()                          # ④ 역전파
    optimizer.step()                         # ⑤ 가중치 수정
```

Pytorch를 이용한 딥러닝 모델 개발

- 분류(Classification) 문제: 모델 설계, 학습, 평가

4) 모델 평가 및 성능 측정

```
model.eval()

with torch.no_grad():
    test_preds = model(X_test)

    # 1. MSE (평균 제곱 오차) 계산
    mse = criterion(test_preds, y_test).item()

    # 2. RMSE (제곱근 평균 제곱 오차): 오차를 원래 단위로 되돌림
    rmse = np.sqrt(mse)

    # 3. 실제 값과 예측값 직접 비교 (상위 3개)
    for i in range(3):
        print(f"실제값: {y_test[i].item():.2f} vs 예측값: {test_preds[i].item():.2f}")
```

MLP 분류 모델 실습

MLP 회귀 모델 실습


MISSION:
COMPLETE